Exploratory Notebook

This notebook shows analysis of data as the following

1- Univariate: Studying each feature alone

2- Bivariate: Studying relationships between two features

3- MultiVariate: Studying relationships between more than two features

First: Loading the data and print insights before studying it

In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json


#  Data Loading
df = pd.read_csv("your_data.csv")

# remove useless column
df = df.drop(columns=["Unnamed: 0"])

df.head()
```

Out[4]:

	fasting blood sugar	age	triglyceride	systolic	relaxation	ALT	HDL	eyesight(left)	weight(kg)	AST	smoking
0	94	55	300	135	87	25	40	0.5	60	22	1
1	147	70	55	146	83	23	57	0.6	65	27	0
2	79	20	197	118	75	31	45	0.4	75	27	1
3	91	35	203	131	88	27	38	1.5	95	20	0
4	91	30	87	121	76	13	44	1.5	60	19	1

In [5]:
```
print(df.shape)

print(df.isnull().sum())
```

```
(159256, 11)
fasting blood sugar    0
age                    0
triglyceride           0
systolic               0
relaxation             0
ALT                    0
HDL                    0
eyesight(left)         0
weight(kg)             0
AST                    0
smoking                0
dtype: int64
```

In [6]:
```
df.describe()
```

Out[6]:

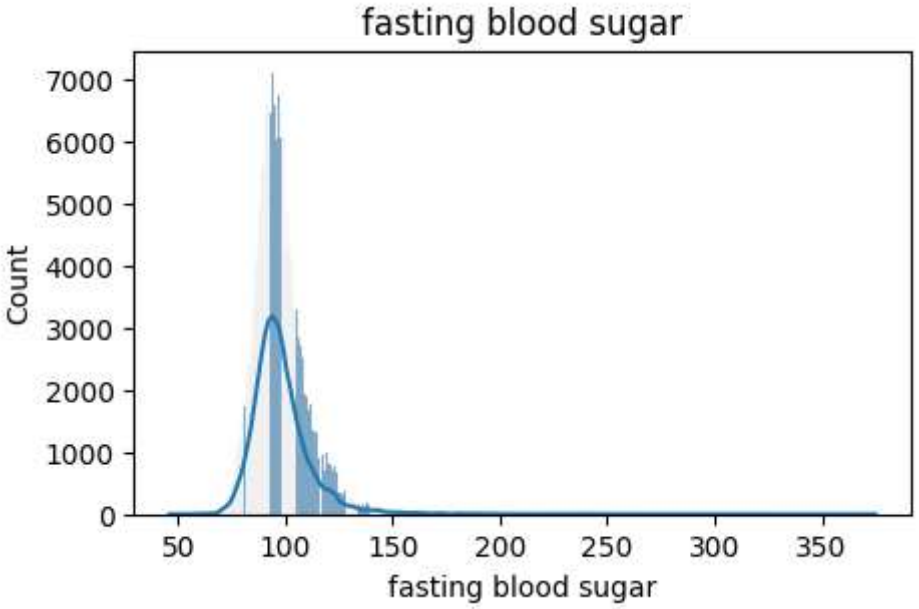
	fasting blood sugar	age	triglyceride	systolic	relaxation	ALT	HDL	eyesight(left)
count	159256.000000	159256.000000	159256.000000	159256.000000	159256.000000	159256.000000	159256.000000	159256.000000
mean	98.352552	44.306626	127.616046	122.503648	76.874071	26.550296	55.852684	1.005798
std	15.329740	11.842286	66.188989	12.729315	8.994642	17.753070	13.964141	0.402113
min	46.000000	20.000000	8.000000	77.000000	44.000000	1.000000	9.000000	0.100000
25%	90.000000	40.000000	77.000000	114.000000	70.000000	16.000000	45.000000	0.800000
50%	96.000000	40.000000	115.000000	121.000000	78.000000	22.000000	54.000000	1.000000
75%	103.000000	55.000000	165.000000	130.000000	82.000000	32.000000	64.000000	1.200000
max	375.000000	85.000000	766.000000	213.000000	133.000000	2914.000000	136.000000	9.900000

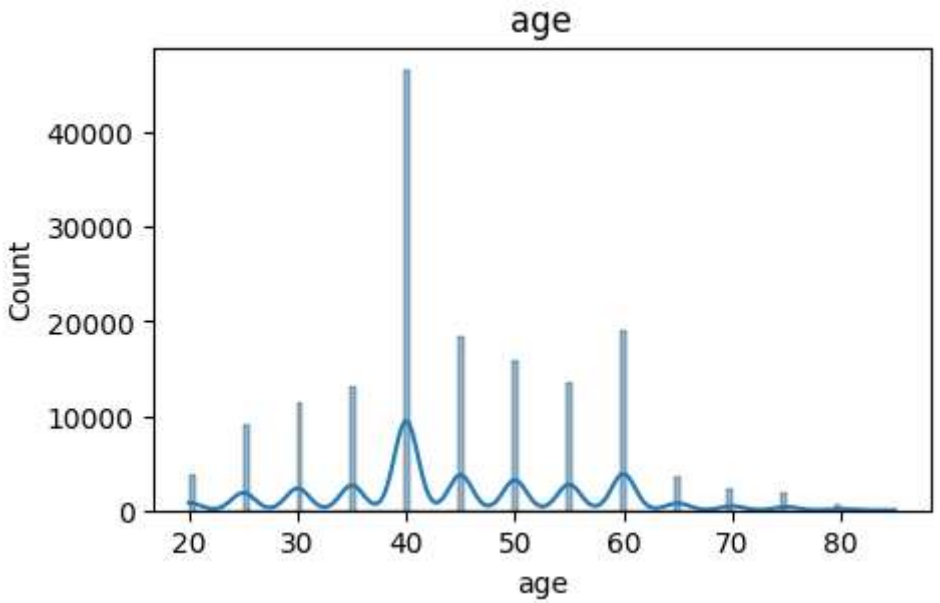
This shows clearly that the data is complete without any missing values; however, some extreme values "*outliers*" exist.

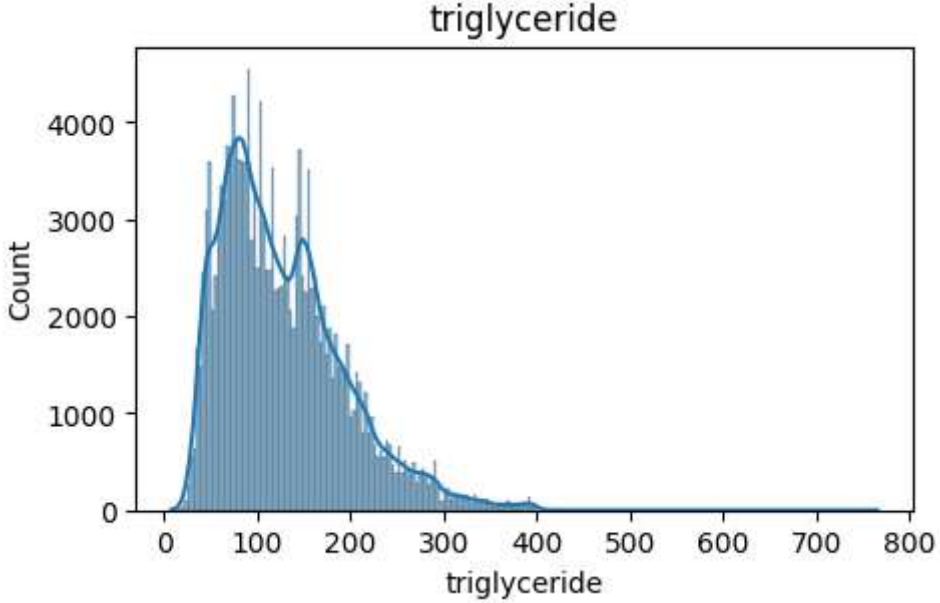
1- Univariate Analysis

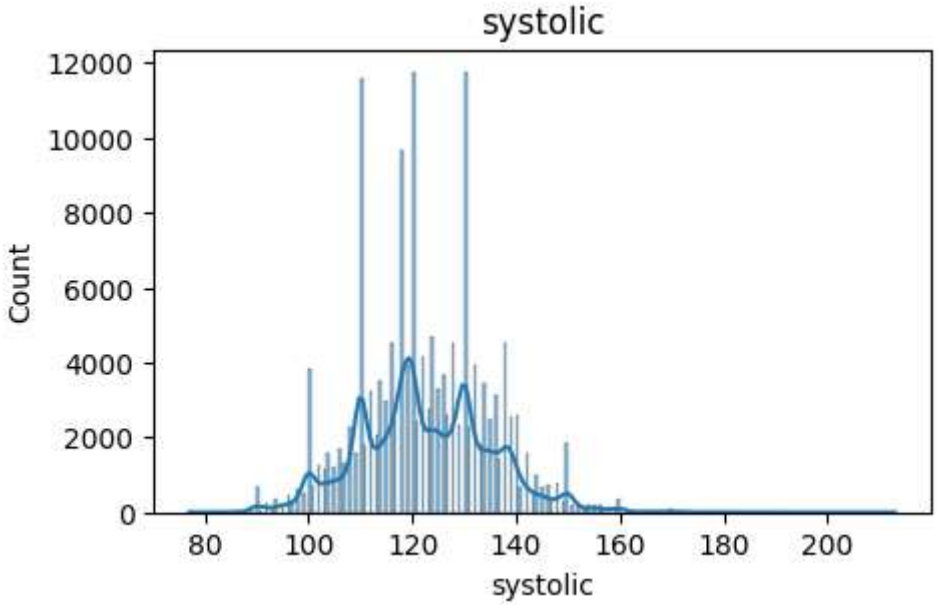
We analyze each feature individually to understand its distribution. This helps identify skewness, spread, and general behavior of each variable in the dataset.

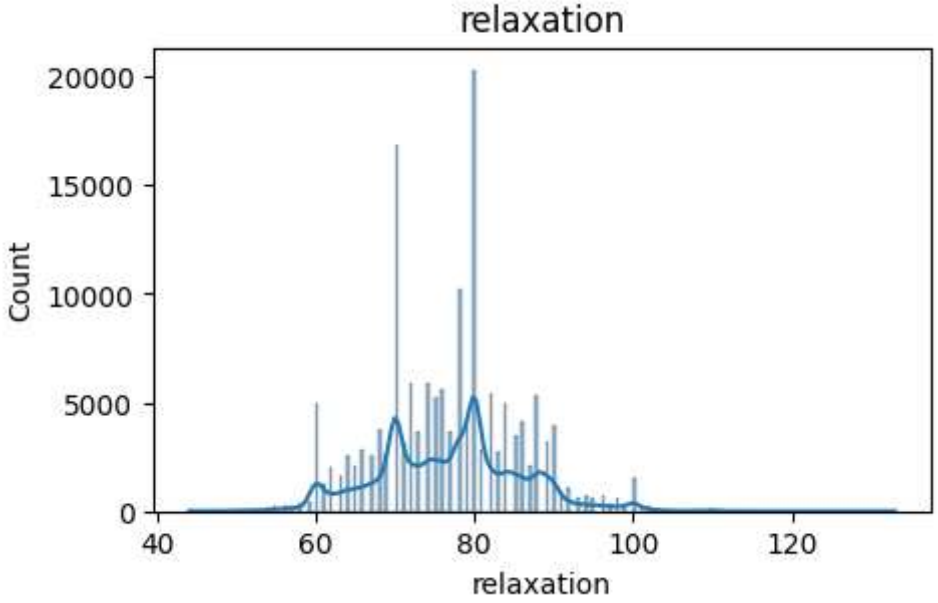
In [7]:
```
for col in df.columns:
    if col != "smoking":
        plt.figure(figsize=(5,3))
        sns.histplot(df[col], kde=True)
        plt.title(col)
        plt.show()
```

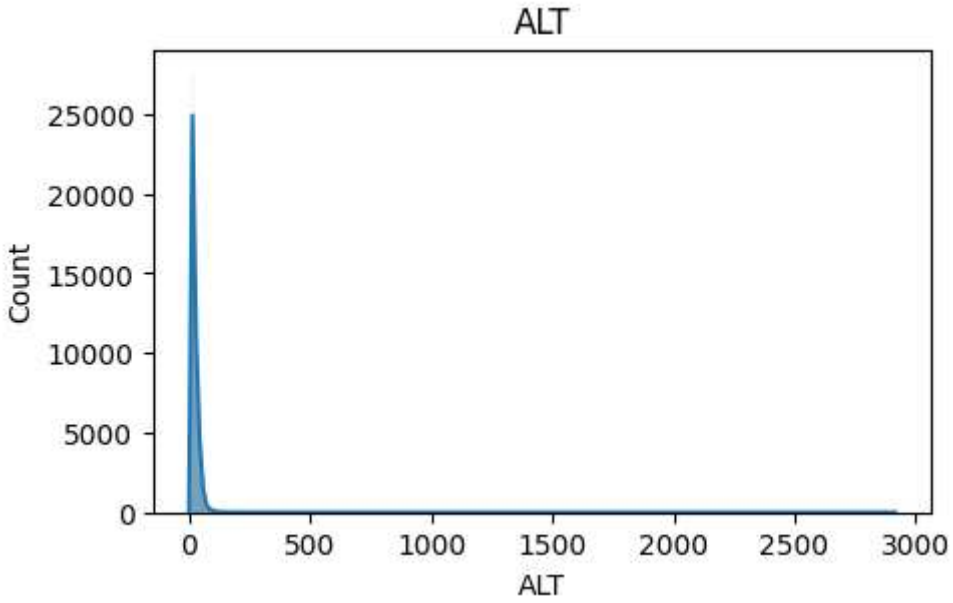


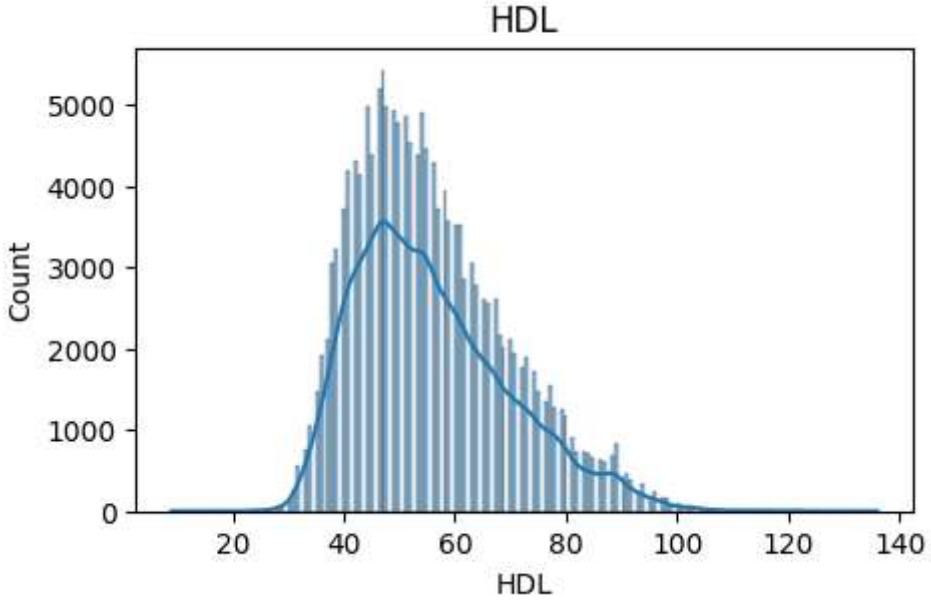

age
Count
40000
30000
20000
10000
0
20
30
40
50
60
70
80
age

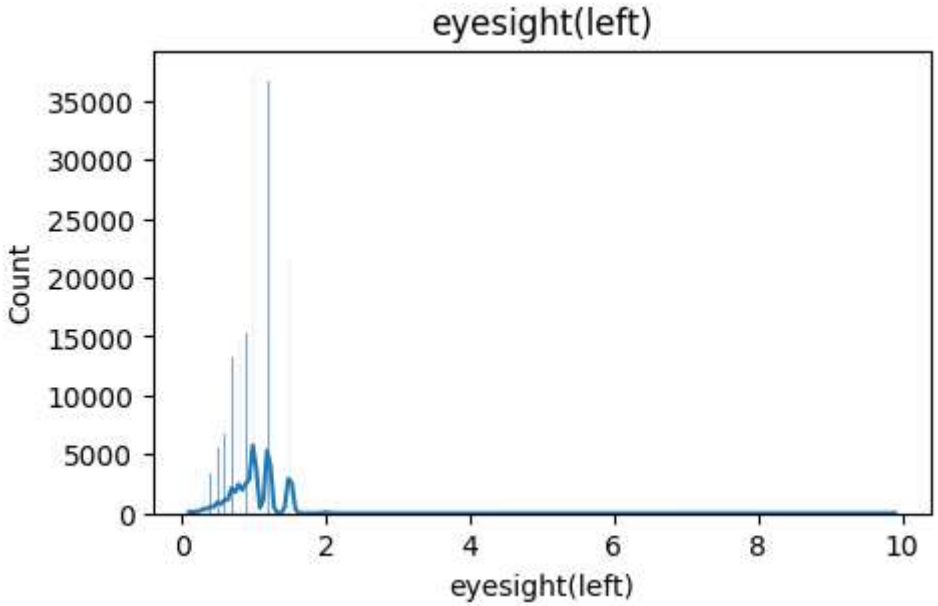

triglyceride
Count
4000
3000
2000
1000
0
0
100
200
300
400
500
600
700
800
triglyceride

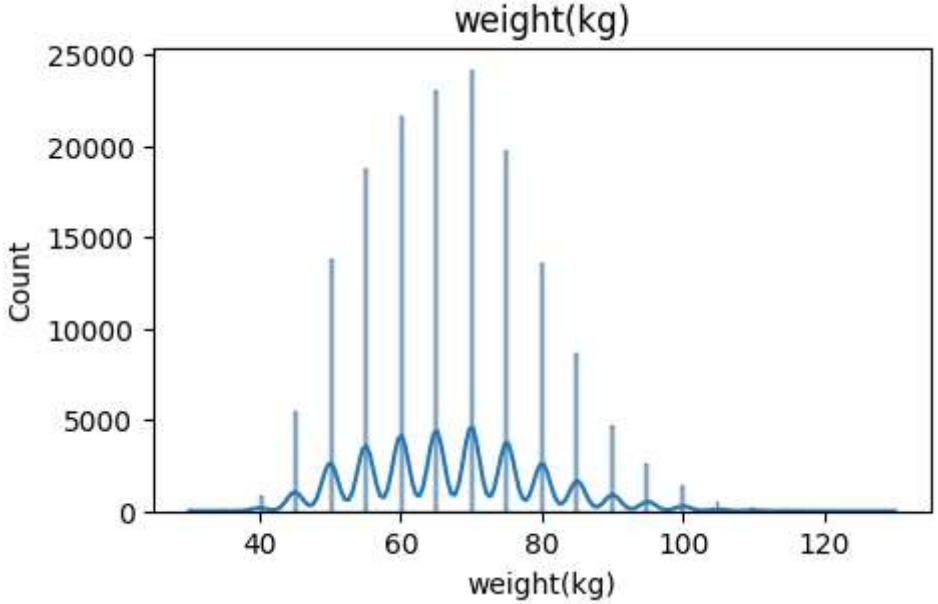

systolic
12000
10000
8000
6000
4000
2000
0
Count
80
100
120
140
160
180
200
systolic

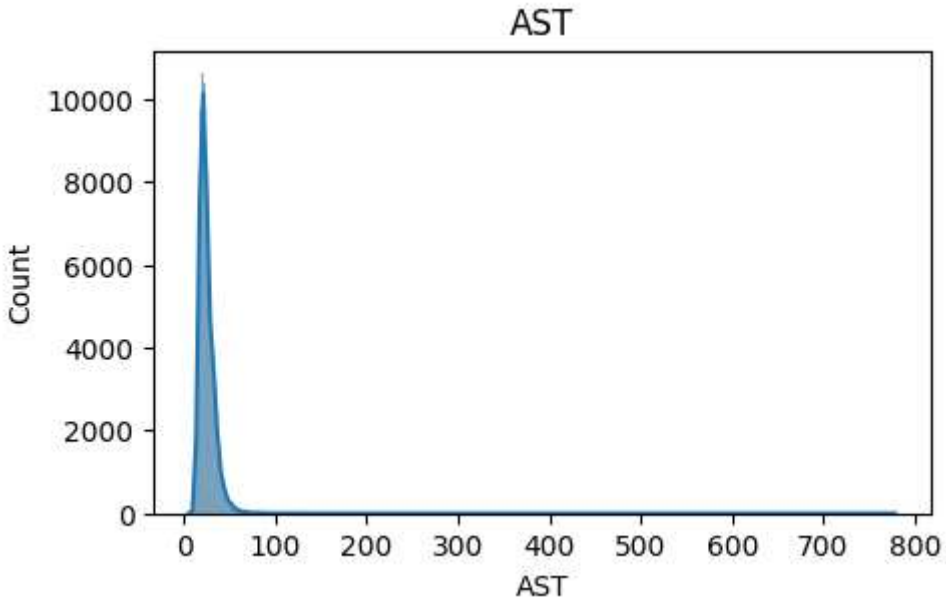

relaxation
20000
15000
10000
5000
0
Count
40
60
80
100
120
relaxation


ALT
25000
20000
15000
10000
5000
0
Count
0
500
1000
1500
2000
2500
3000
ALT


HDL
5000
4000
3000
2000
1000
0
Count
20
40
60
80
100
120
140
HDL


eyesight(left)
Count
35000
30000
25000
20000
15000
10000
5000
0
0
2
4
6
8
10
eyesight(left)

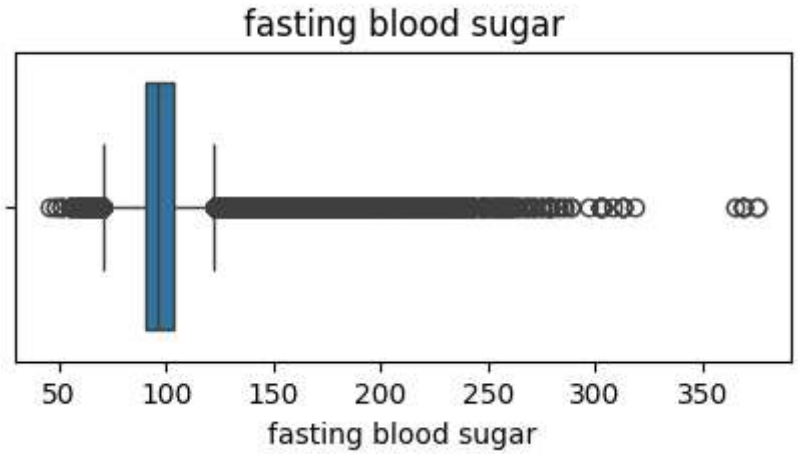

weight(kg)
Count
25000
20000
15000
10000
5000
0
40
60
80
100
120
weight(kg)

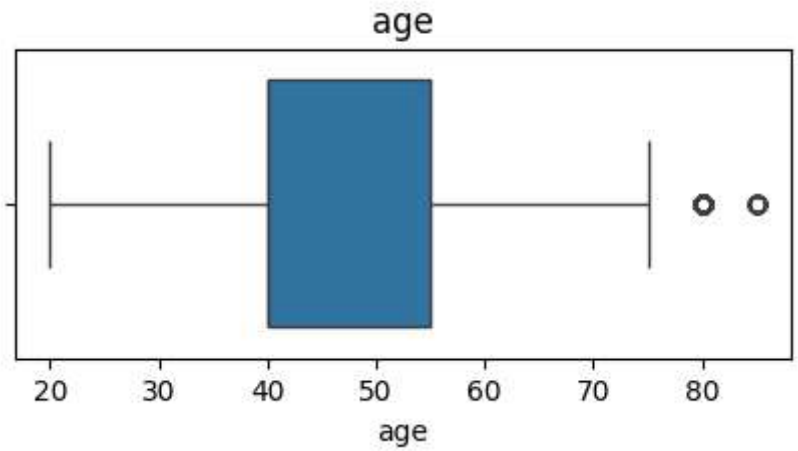


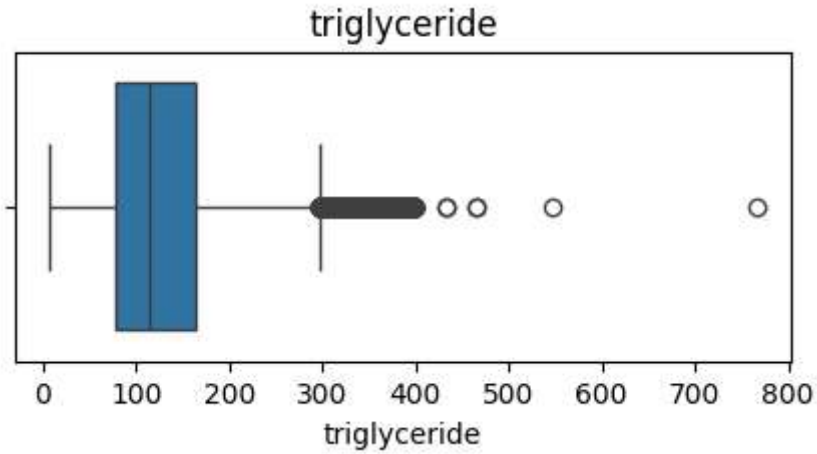
Outlier Detection

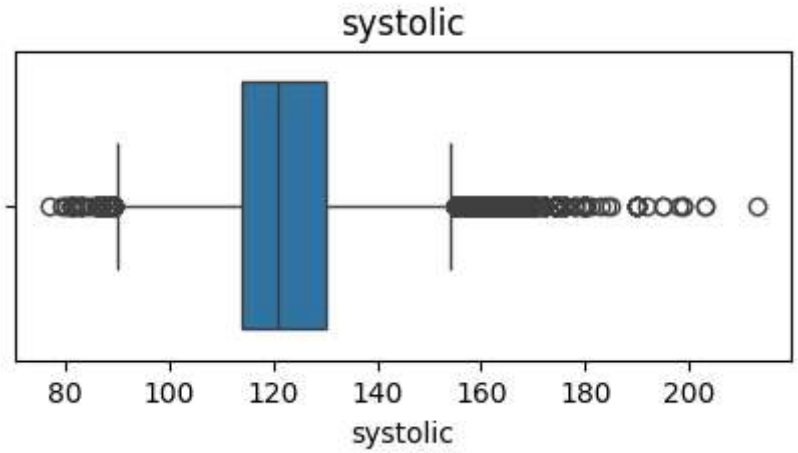
Boxplots are used to detect outliers in each feature. Outliers are extreme values that may affect model performance and need to be handled during preprocessing.

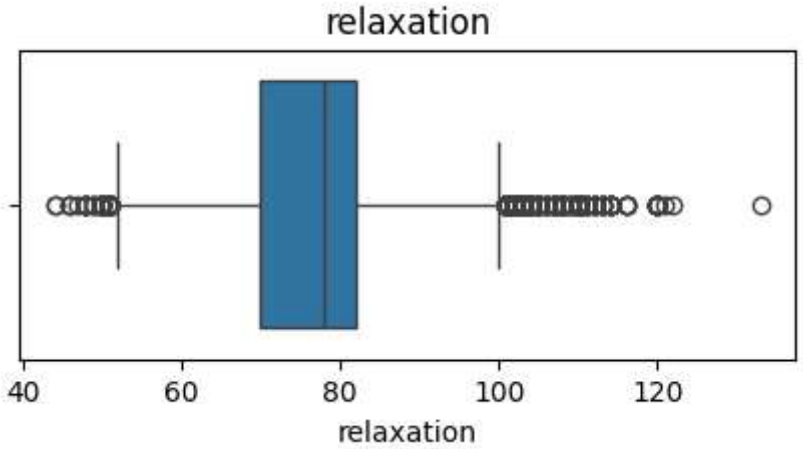
In [8]:
```
for col in df.columns:
    if col != "smoking":
        plt.figure(figsize=(5,2))
        sns.boxplot(x=df[col])
        plt.title(col)
        plt.show()
```

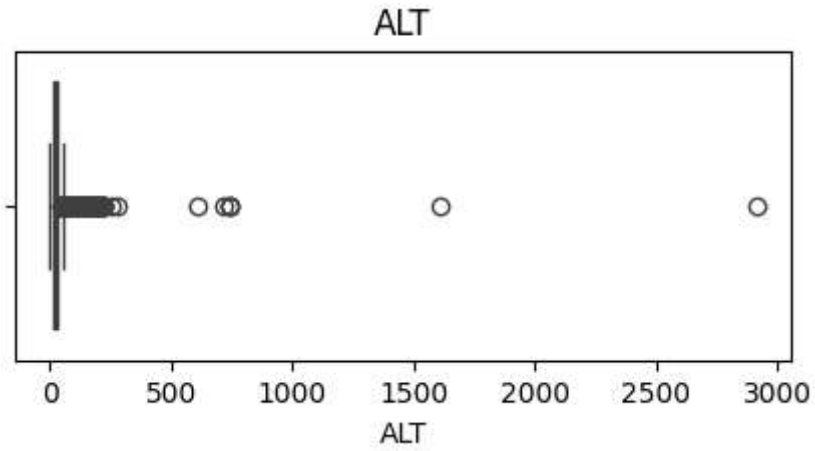

fasting blood sugar
50
100
150
200
250
300
350
fasting blood sugar

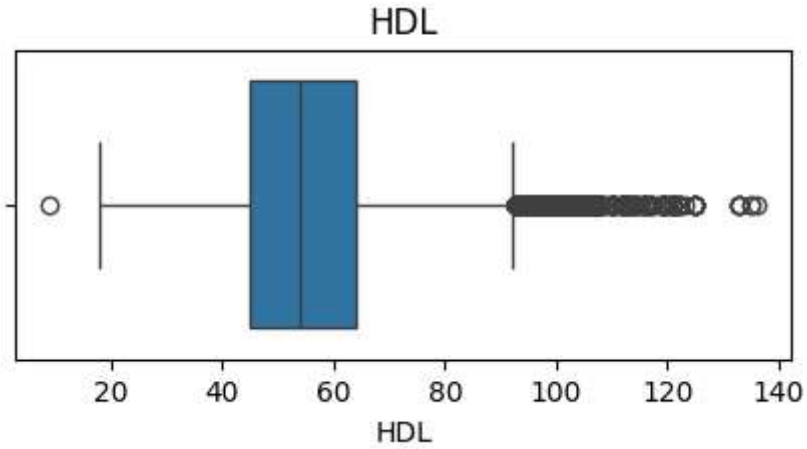

age
20
30
40
50
60
70
80
age

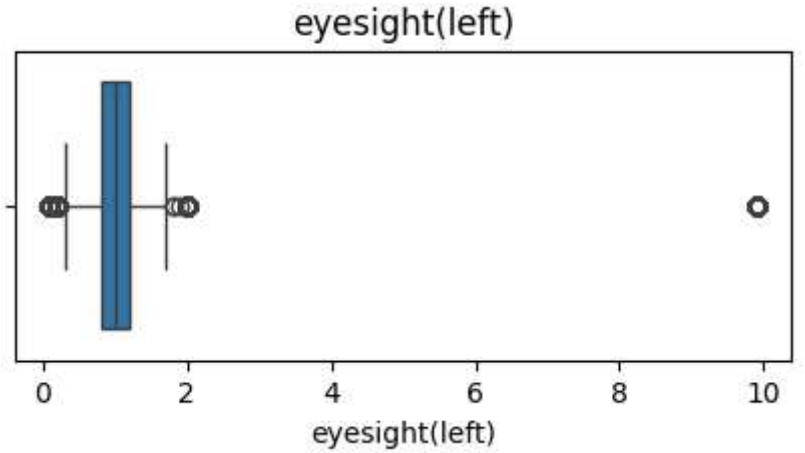

triglyceride
0
100
200
300
400
500
600
700
800
triglyceride

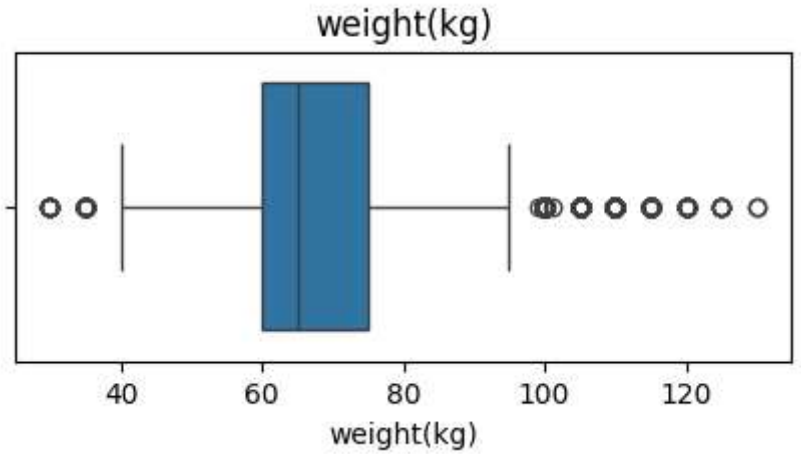

systolic
80
100
120
140
160
180
200
systolic

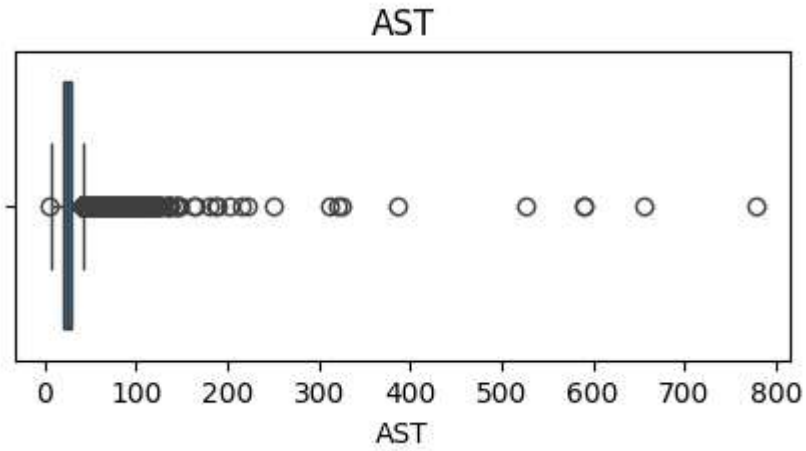

relaxation
40
60
80
100
120
relaxation


ALT
0
500
1000
1500
2000
2500
3000
ALT


HDL
20
40
60
80
100
120
140
HDL


eyesight(left)
0
2
4
6
8
10
eyesight(left)

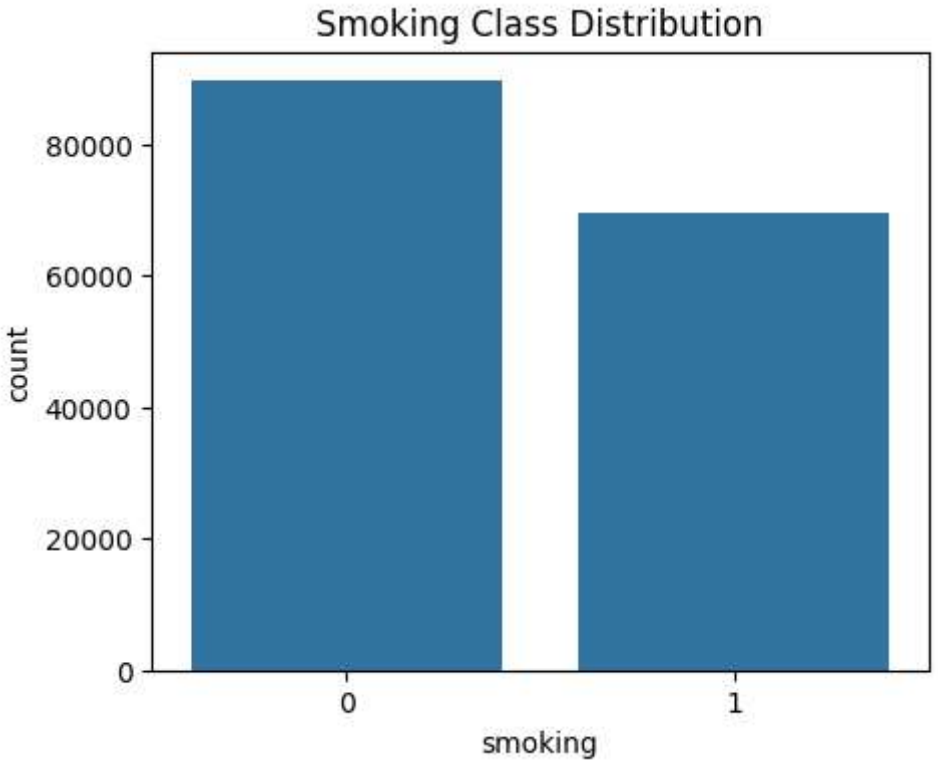




Smoking distrubution

To ensure that no dominant class and that the target data is balanced

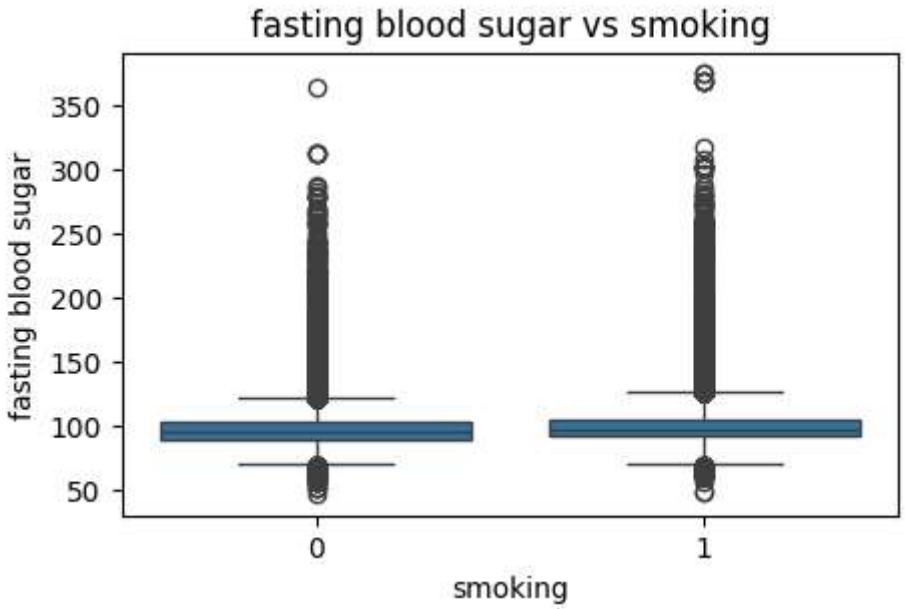
In [9]:
```
plt.figure(figsize=(5,4))

sns.countplot(x=df["smoking"])

plt.title("Smoking Class Distribution")

plt.show()
```

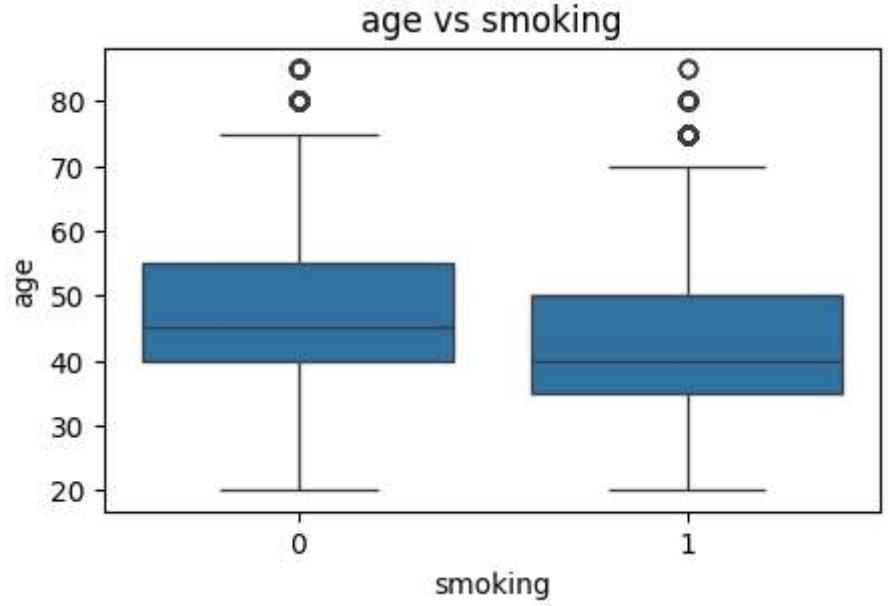


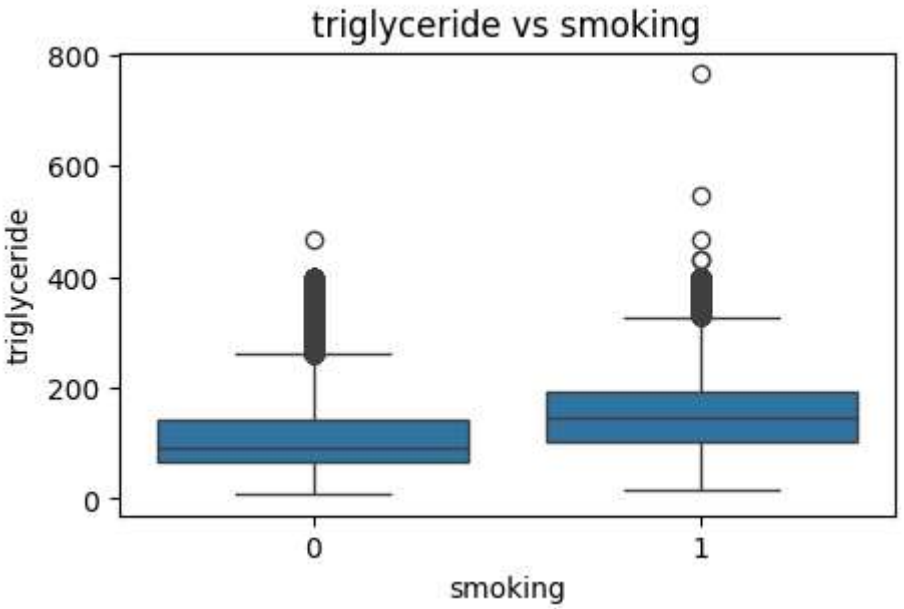
Bivariate Analysis

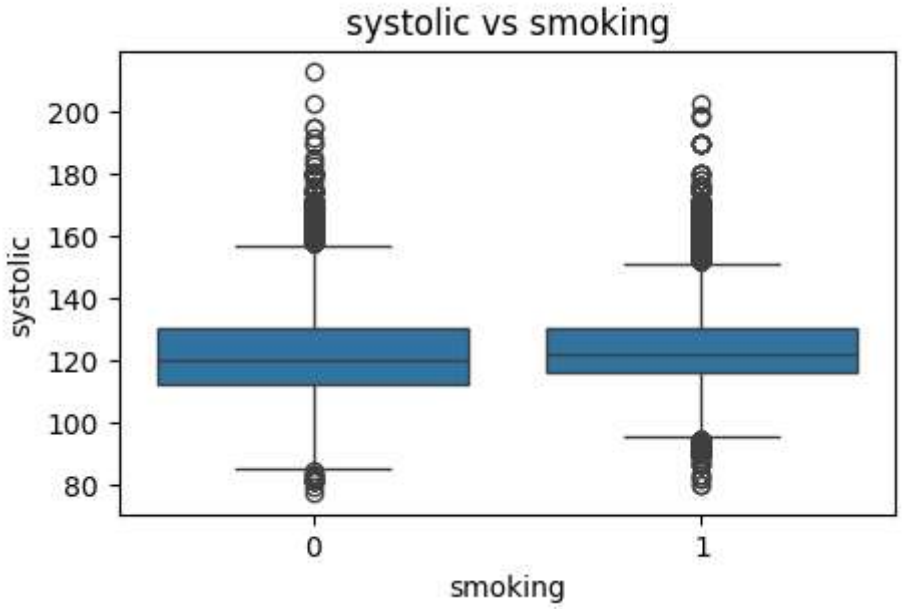
We study the relationship between each feature and the target variable (smoking). This helps identify how different variables vary between smokers and non-smokers.

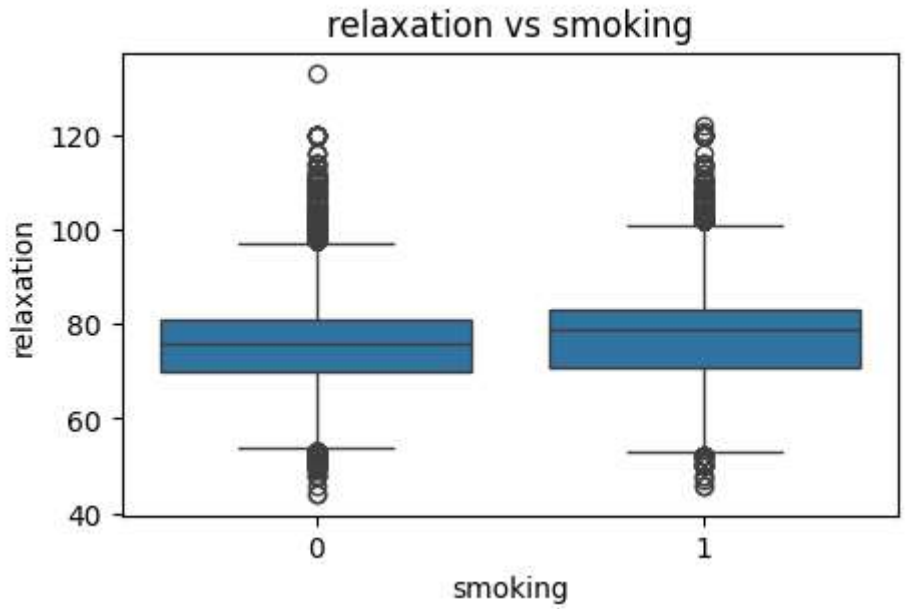
In [10]:
```
for col in df.columns:
    if col != "smoking":
        plt.figure(figsize=(5,3))
        sns.boxplot(x="smoking", y=col, data=df)
        plt.title(f"{col} vs smoking")
        plt.show()
```

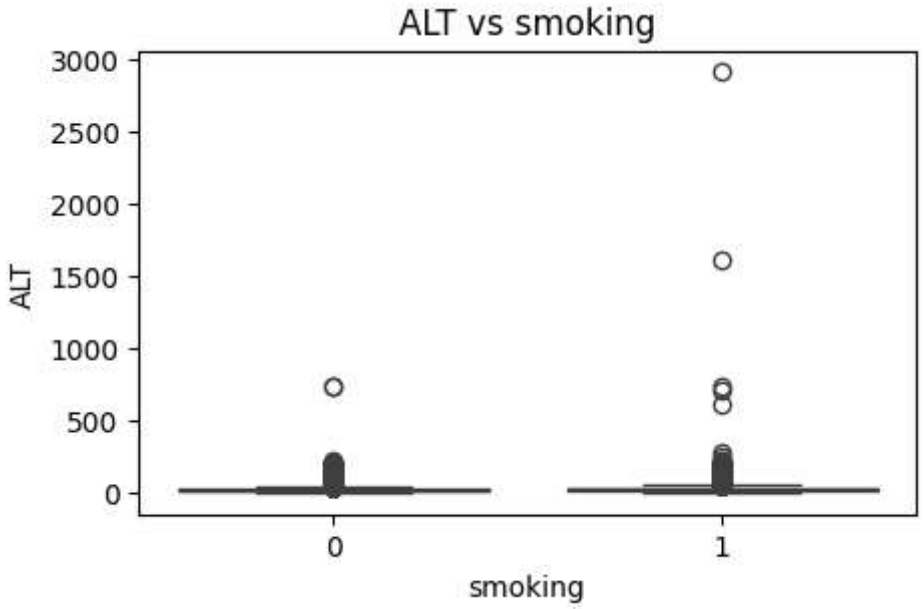

fasting blood sugar vs smoking
fasting blood sugar
350
300
250
200
150
100
50
0
1
smoking

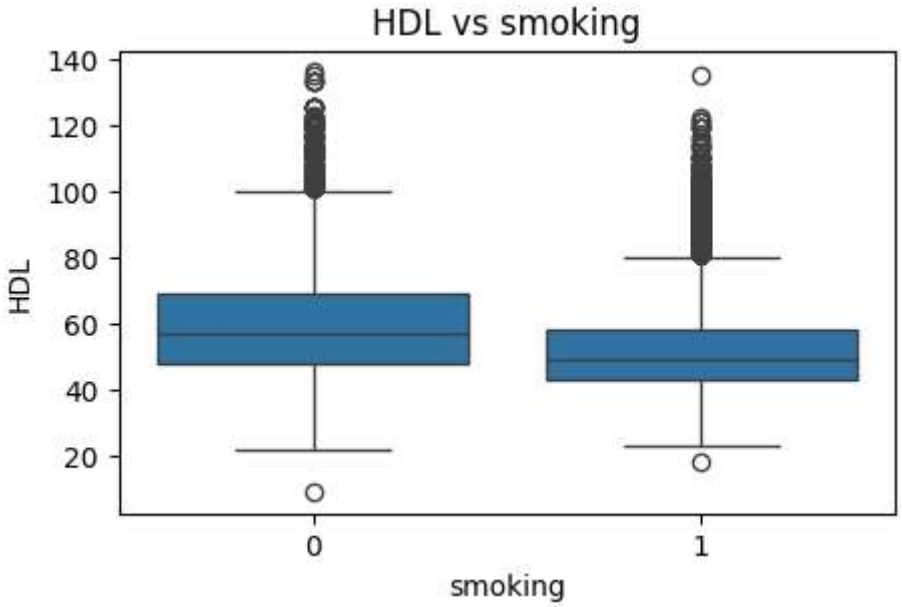

age vs smoking
age
80
70
60
50
40
30
20
0
1
smoking

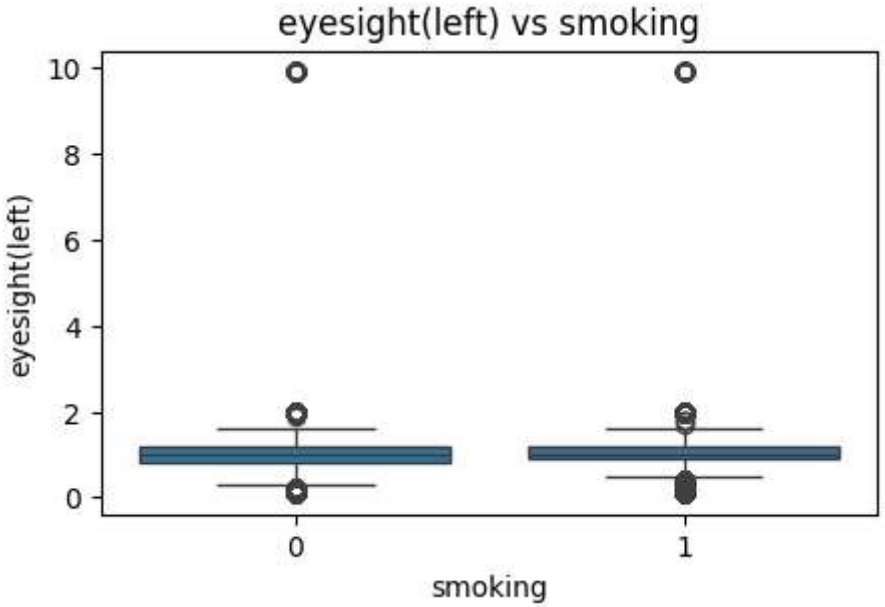

triglyceride vs smoking
triglyceride
800
600
400
200
0
0
1
smoking

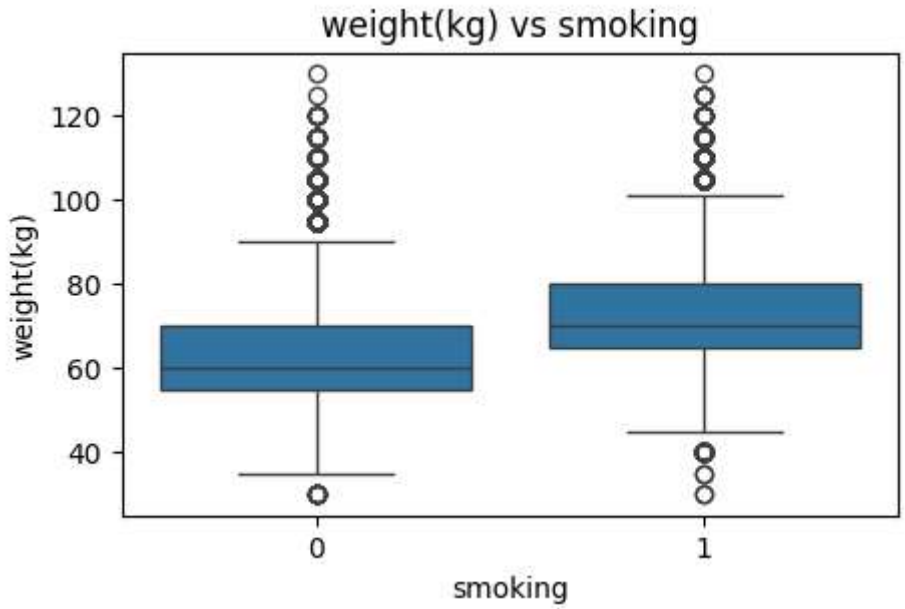

systolic vs smoking
systolic
200
180
160
140
120
100
80
0
1
smoking

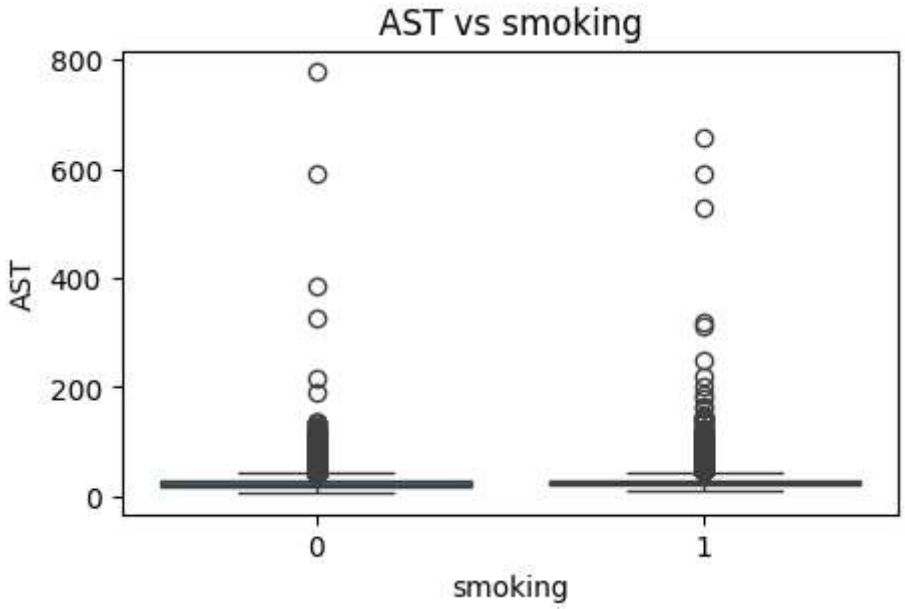

relaxation vs smoking
relaxation
120
100
80
60
40
0
1
smoking


ALT vs smoking
ALT
3000
2500
2000
1500
1000
500
0
0
1
smoking


HDL vs smoking
HDL
140
120
100
80
60
40
20
0
1
smoking


eyesight(left) vs smoking
eyesight(left)
10
8
6
4
2
0
0
1
smoking

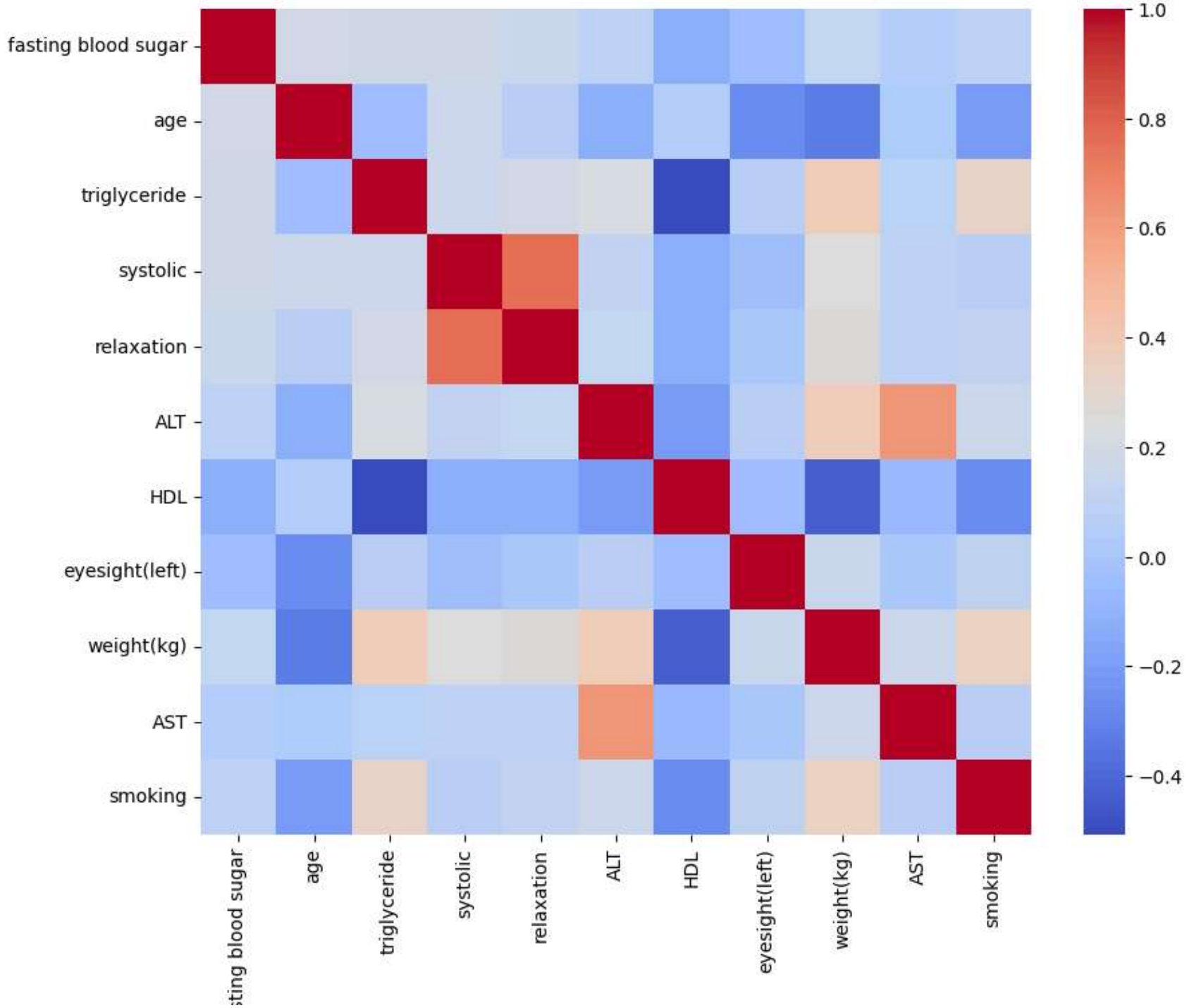




Correlation Analysis

A correlation heatmap is used to measure relationships between numerical features. It helps identify highly correlated variables that may provide similar information.

In [11]:
```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
plt.show()
```

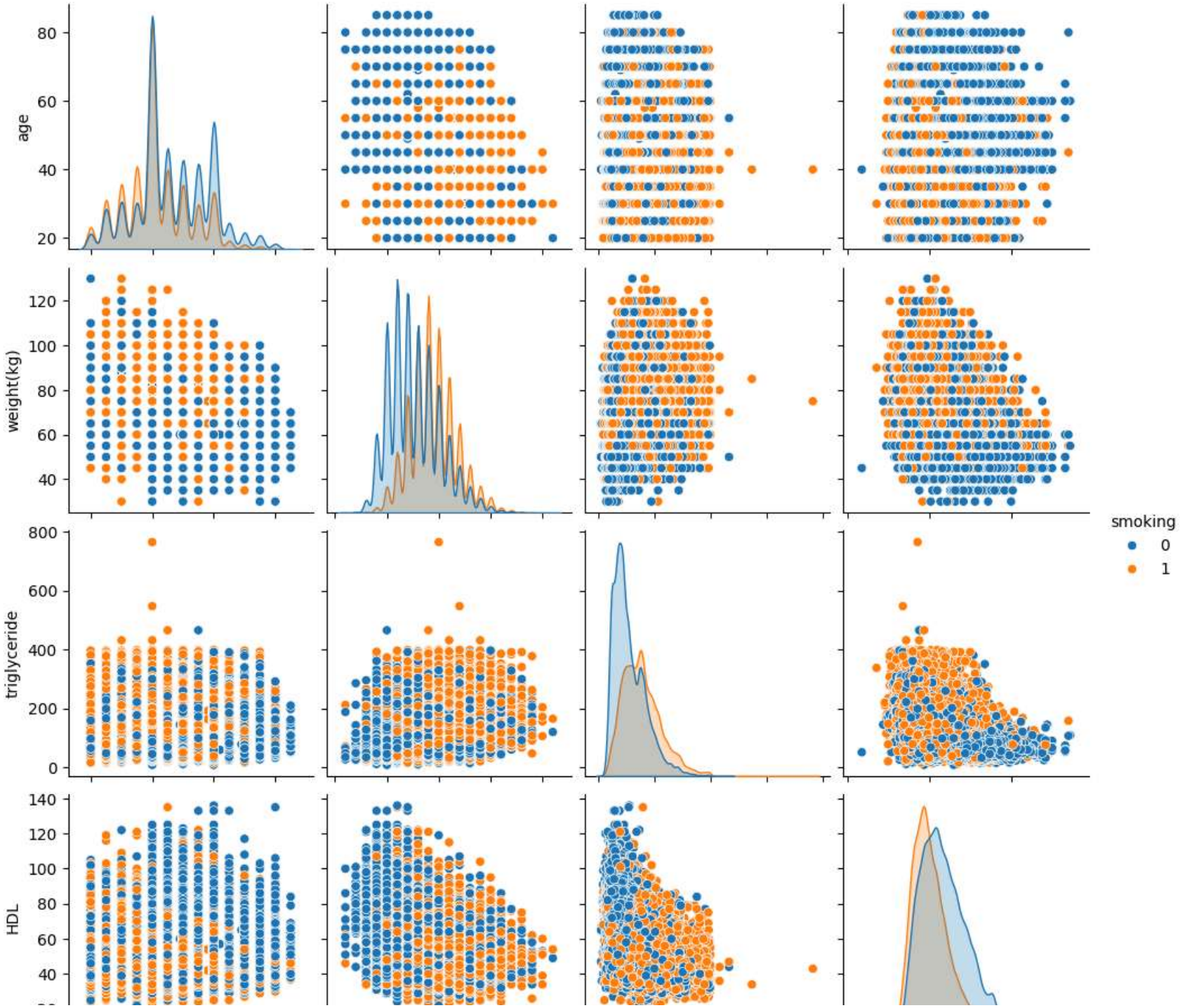

fasting blood sugar
age
triglyceride
systolic
relaxation
ALT
HDL
eyesight(left)
weight(kg)
AST
smoking
:ting blood sugar
age
triglyceride
systolic
relaxation
ALT
HDL
eyesight(left)
weight(kg)
AST
smoking
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

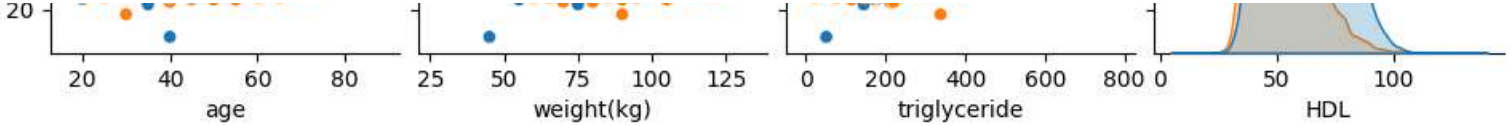
fas

Multivariate Analysis

We use pairplots to visualize relationships between multiple features simultaneously. This helps in understanding complex interactions between variables.

In [12]:
```
sns.pairplot(df[["age","weight(kg)","triglyceride","HDL","smoking"]],
             hue="smoking")
plt.show()
```


age
weight(kg)
triglyceride
HDL
smoking
0
1



Outlier Removal

We apply the IQR method to remove extreme values from the dataset. This improves model stability and prevents bias caused by abnormal data points.

In [13]:
```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)

IQR = Q3 - Q1

df = df[~((df < (Q1 - 1.5 * IQR)) |
          (df > (Q3 + 1.5 * IQR))).any(axis=1)]
```

In [14]:
```
X = df.drop("smoking", axis=1)
y = df["smoking"]
```

Feature Scaling

We apply standardization to normalize feature values to the same scale. This ensures that all features contribute equally to the model.

In [15]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Data Splitting

The dataset is split into training and testing sets. The training set is used to build the model, while the testing set is used to evaluate its performance.

In [16]:
```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42
)
```

Logistic Regression Model

A logistic regression model is trained as a simple linear classification approach. It models the probability of smoking based on input features.

In [17]:
```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
```

Out[17]:

▾ LogisticRegression

▸ Parameters

Model Evaluation

We evaluate the model using accuracy, precision, recall, and F1-score. These metrics help assess the model's classification performance from different perspectives.

In [18]:
```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))
```

```
Accuracy: 0.690803241885773
Precision: 0.649168243848989
Recall: 0.5860084524772952
F1-score: 0.6159735349716446
```

Feature Importance

The coefficients of the logistic regression model are analyzed to understand feature importance. Positive values increase the likelihood of smoking, while negative values decrease it.

In [19]:
```python
importance = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0]
})

importance = importance.sort_values(by="Coefficient", ascending=False)
top_features = importance.head(4)["Feature"].tolist()
importance
with open("top_features.json", "w") as f:
    json.dump(top_features, f)
```